

COCO: A Cohesiveness-aware Learning Framework for Community Search over Temporal Graphs

Anonymous Author(s)

Abstract

In this paper, we explore time-interval-based community search over temporal graphs, which aims to identify a connected cohesive subgraph containing the query vertices within a specific time interval. Existing work suffers from two main limitations: (1) *information loss*, as it ignores the interaction frequency between vertices, a crucial factor for identifying high-quality communities. (2) *efficiency bottleneck*, as it either incurs high search latency from expensive online computation or faces a combinatorial explosion in indexing time and storage when precomputing coreness values across all possible time intervals. To address these issues, we propose COCO, a cohesiveness-aware learning framework for time-interval-based community search over temporal graphs. COCO works in three phases: pre-training, fine-tuning, and searching. It contains two novel components: (1) a *hierarchical model index*, which efficiently computes coreness across different time intervals via a hierarchical model tree, and (2) an *adaptive temporal GNN*, which effectively captures both graph structural information and interaction frequencies while supporting efficient fine-tuning. Experiments on eight real-world datasets show that COCO outperforms state-of-the-art methods in community quality and is at least 10× faster than the baselines, demonstrating its effectiveness and efficiency.

1 Introduction

Community search is an important tool in graph analysis, aiming to identify cohesive subgraphs that contain given query vertices [7, 12, 26, 40]. Prior research has extensively investigated community search on various graph types, including directed graphs [3, 8, 22], bipartite graphs [37, 38], attributed graphs [23, 41], and multilayer graphs [2, 24, 33].

In real-world applications, interactions between entities are dynamic and evolve over time. For example, in social networks, user activities such as comments, likes, and shares are timestamped, which reflects the temporal nature of relationships. This temporal information provides an important view for understanding user behavior patterns and community structure. As a result, community search over temporal graphs has attracted considerable research attention [18–21, 43, 46]. Among these studies, [43] investigates community search within specified time intervals on temporal graphs. Given a temporal graph, a query vertex, a time interval, and an integer k , the method in [43] returns a connected k -core

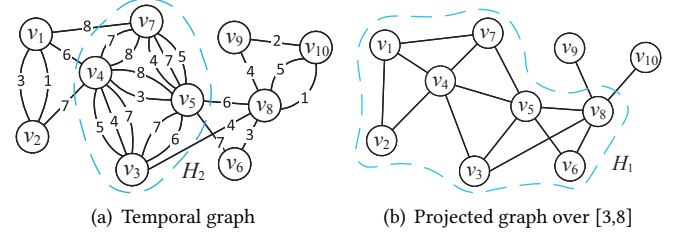


Figure 1: A motivating example

containing the query vertex in a projected graph. This projected graph is constructed by retaining only the edges within the specified time interval, where multiple edges between the same pair of vertices are merged into one. For example, Figure 1(a) shows a temporal graph in which each edge is timestamped to indicate when the corresponding interaction occurred. Given the query vertex v_4 , the time interval $[3, 8]$, and $k = 2$, the method in [43] identifies community H_1 in the corresponding projected graph, as shown in Figure 1(b).

Limitations. The existing method for time-interval-based community search over temporal graphs suffers from two major limitations.

L1. Information Loss. The method in [43] identifies communities directly in the projected graph for a given time interval. However, this projected graph fails to capture the interaction frequency between vertices. Interaction frequency (the number of distinct temporal edges between two vertices) is a direct measure of connection strength. Groups with higher interaction frequencies signify tighter relationships and thus form more cohesive communities. Ignoring this frequency leads to overlooking densely connected subgraphs, which are essential for identifying high-quality communities. For example, in Figure 1(a), H_2 is a better community for v_4 within the time interval $[3, 8]$, as each vertex in H_2 interacts with its neighbors an average of 6 times, indicating a more cohesive group.

L2. Efficiency Bottleneck. The method in [43] has a serious efficiency limitation. Performing online core decomposition for each query interval is computationally expensive and leads to high search latency on large-scale graphs. For example, online core decomposition on a graph with millions of vertices may take around 500 seconds (see Section 6.2 for details). Although indexing can accelerate queries, it requires precomputing and storing results for all possible k values and time intervals. Since a temporal graph with $t_{\max}$ timestamps contains $O(t_{\max}^2)$ continuous intervals, this approach incurs prohibitive preprocessing time and memory overhead, making it impractical for large-scale temporal graphs. In particular, the index for a graph with two million vertices can exceed 1 GB, and for even larger graphs, it cannot be constructed within 24 hours (see Section 6.2 for details).

To address these limitations, we revisit and formalize the *Time-Interval-based Community Search (TICS)* problem. Specifically, given a temporal graph $\mathcal{G}$, a time interval $[t_s, t_e]$, and a set of query

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD '27, Huntington Beach, CA, USA

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

vertices V_Q , the goal of TICS is to identify a community containing V_Q within $[t_s, t_e]$.

Challenges. The TICS problem presents two main challenges.

C1. How to identify high-quality communities? High-quality temporal communities should exhibit both cohesive structural topology and frequent temporal interactions. However, integrating these two heterogeneous signals is intrinsically difficult. Existing structure-based cohesive subgraphs cannot effectively measure interactions frequency, while purely frequency-based metrics often overlook the overall specific community structures.

C2. How to address TICS efficiently? The time interval of each TICS query may differ, and therefore the graph topology associated with each query can also be different. Processing every TICS query from scratch is computationally expensive, especially on large-scale temporal graphs. Although preprocessing or indexing may help, such methods must handle the large number of possible graph structures across all time intervals, which further increases the computational cost.

Our Solutions. To overcome these challenges, we propose **COCO**, a novel **C**ohesiveness-aware learning framework for **C**ommunity search over temporal graphs. COCO employs a three-stage design (pre-training, fine-tuning, and searching) that seamlessly integrates the classical k -core with graph neural networks (GNNs). In this design, the k -core supplies structural information, enabling the GNN to jointly encode both cohesiveness and interaction frequencies. COCO offers two key advantages: (1) it effectively identifies high-quality communities with both strong structural cohesiveness and frequent interactions; and (2) it supports fast query processing over arbitrary time intervals without recomputation.

Specifically, COCO incorporates two components. (1) The **Hierarchical Model Index (HM-Index)** supports efficient vertex coreness prediction over arbitrary time intervals (**for C2**). It avoids explicit core decomposition, which is computationally expensive. It recursively partitions the temporal span into a hierarchical tree, equipping each node with a lightweight MLP. Instead of making a one-step prediction, the HM-Index uses a progressive prediction scheme. For a query interval, it traverses the tree from the root, and parent nodes iteratively provide coarse estimates that guide the computations of child nodes along the path. This design avoids the large storage cost of indexing all subintervals while providing accurate, real-time coreness estimation. (2) The **Adaptive Temporal GNN (AT-GNN)** is specifically designed to capture both structural cohesiveness and vertex interaction frequencies (**for C1**). By integrating the coreness values from the HM-Index with interaction frequencies, AT-GNN significantly enhances its ability to identify high-quality communities. Furthermore, the AT-GNN incorporates an adapter module, which allows it to rapidly fine-tune and adapt to the unique graph structures of different queries without requiring a full retraining process (**for C2**).

The COCO operates through a three-phase pipeline. First, in the pre-training phase, COCO trains coreness prediction models on different time intervals and then organizes these models into an HM-Index; and trains AT-GNN in an unsupervised manner, enabling it to learn common community patterns across time intervals. In the fine-tuning phase, COCO predicts the coreness of query vertices and their neighbors by traversing the HM-Index, and uses these

coreness values along with the query context to adjust the adapter module parameters of AT-GNN. In the searching phase, COCO uses vertex embeddings from AT-GNN to iteratively select vertices most similar to the query vertices, forming the final community.

Contributions. Overall, our main contributions are as follows.

- We propose COCO, a novel cohesiveness-aware learning framework for time-interval-based community search (TICS). It combines the classic k -core with GNNs and can efficiently identify high-quality communities in large temporal graphs.
- We design the *Hierarchical Model Index* (HM-Index), a learning-based component for efficiently prediction vertex coreness over any time interval. It arranges lightweight models in a hierarchical tree, eliminating the high preprocessing time and storage costs of traditional indexing methods.
- We develop the *Adaptive Temporal GNN* (AT-GNN) to capture graph structure and temporal interaction frequencies. It includes a query-aware adapter module, allowing quick adaptation to different query structures without retraining.
- We conduct extensive experimental studies on eight real-world graphs. The results validate the superior effectiveness and efficiency of our proposed COCO framework.

2 Related Work

We review the related work from two aspects, including classic community search over temporal graphs, and learning-based methods for community search.

Classic community search over temporal graphs. Based on temporal graph characteristics, various community search methods have been studied. For instance, time-interval-based approaches [18, 43, 46] aim to find communities in the projected graph over a given time interval. [19, 20] assume graphs are snapshots and require communities to exist in every snapshot within the interval. Others, like [21], focus on query-centered search by combining temporal proximity and structural cohesion. Several cohesive subgraph models have also been proposed, including span core [9, 10], periodic clique [27], periodic core [29], temporal k -core [42, 44, 47], and bursting core [28]. However, these do not support query-based search. In this work, we focus on time-interval-based community search and propose COCO, a new framework that overcomes limitations of prior methods discussed in Introduction.

Learning-based methods for community search. The existing learning-based community search methods can be mainly divided into three categories: supervised methods, semi-supervised methods, and unsupervised methods. The supervised methods [6, 13, 36] depend on ground-truth communities, which are used to train models to learn community structures. The semi-supervised methods [17, 32] rely on only a fraction of labeled vertices instead of complete ground-truth labels. The unsupervised methods [35, 39] rely on contrastive learning or heuristic rules to produce pseudo-labels for pretraining, circumventing the requirement for ground-truth communities. Existing learning-based methods struggle with generalization in the TICS problem. They work well on static graphs but fail on dynamic or unseen graph structures.

Recently, Hashem et al. [11] proposed CS-TGN for community search based on the temporal GNN. Zhang et al. [45] proposed UTCS, an unsupervised method for temporal community search.

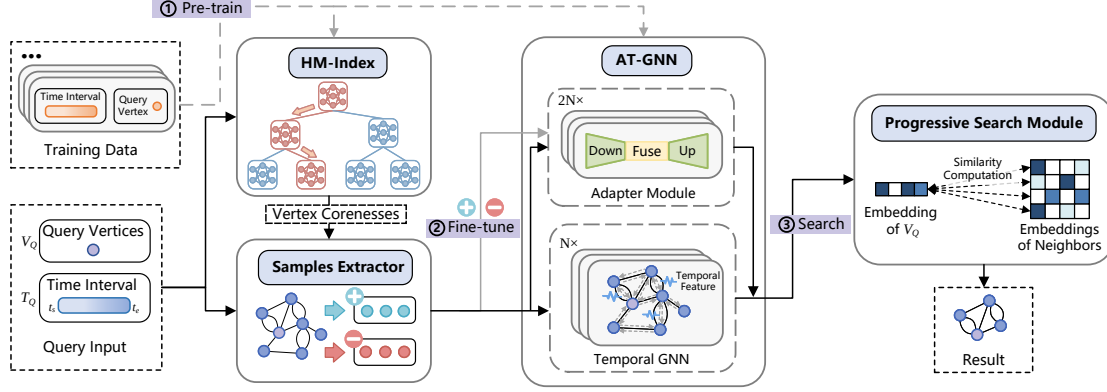


Figure 2: The design of COCO framework

Jiang et al. [14] devised PK-GCN to identify the persistent k -core in the temporal graph based on GNN. Compared to COCO, CS-TGN and UTCS have limited ability to capture temporal information, and ignores the importance of community cohesiveness. PK-GCN still encounters the structure inflexibility problem, since it essentially uses deep learning models to find k -core structures.

3 Preliminary

A temporal graph is represented as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ and $\mathcal{E}$ denote the sets of vertices and temporal edges, respectively. Each edge $e \in \mathcal{E}$ is a triplet (u, v, t) , representing an interaction between vertices u and v at timestamp t . The entire time interval of $\mathcal{G}$ is $\mathcal{T} = [1, t_{\max}]$. For a given time interval $[t_s, t_e] \subseteq \mathcal{T}$, the temporal graph $\mathcal{G}$ over $[t_s, t_e]$ is denoted by $\mathcal{G}_{[t_s, t_e]} = (\mathcal{V}_{[t_s, t_e]}, \mathcal{E}_{[t_s, t_e]})$, where $\mathcal{V}_{[t_s, t_e]} = \mathcal{V}$ and $\mathcal{E}_{[t_s, t_e]} = \{(u, v, t) \mid (u, v, t) \in \mathcal{E}, t \in [t_s, t_e]\}$. The projected graph of $\mathcal{G}$ over $[t_s, t_e]$ is denoted by $\mathcal{G}_{pro[t_s, t_e]} = (\mathcal{V}_{pro[t_s, t_e]}, \mathcal{E}_{pro[t_s, t_e]})$. Here, $\mathcal{V}_{pro[t_s, t_e]} = \mathcal{V}$ and $\mathcal{E}_{pro[t_s, t_e]} = \{(u, v) \mid (u, v, t) \in \mathcal{E}, t \in [t_s, t_e]\}$. We use $N(v)$ to denote the neighbors of vertex v in $\mathcal{G}$. For the neighbors of v over a specific time interval $[t_s, t_e]$, we denote this set as $N_{[t_s, t_e]}(v)$. The degree of v at a particular timestamp $t \in \mathcal{T}$ is denoted by $d_t(v)$. Additionally, the degree of v over the time interval $[t_s, t_e]$ is $d_{[t_s, t_e]}(v) = |N_{[t_s, t_e]}(v)|$. Based on above notations, we formally define our problem.

Problem 1. (Time-Interval-based Community Search (TICS)). Given a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, a time interval $T_Q = [t_s, t_e]$, and a set of query vertices $V_Q \subseteq \mathcal{V}$, the time-interval-based community search aims to find a vertex set C , satisfying:

- (1) $V_Q \subseteq C$;
- (2) The subgraph induced by C in $\mathcal{G}_{[t_s, t_e]}$ is connected;
- (3) Vertices in C are densely intra-connected in $\mathcal{G}_{[t_s, t_e]}$.

Note that “densely intra-connected” in condition (3) requires a community to jointly exhibit strong structural cohesiveness and intensive temporal interaction frequency, as discussed in L1 and C1 in the Introduction. In other words, vertices in the community should not only have many neighbors, but also interact frequently with those neighbors. Next, we introduce several key concepts used in the rest of the paper.

Definition 3.1. (Temporal k -core [43]). Given a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and a time interval $[t_s, t_e]$. Let $\mathcal{G}_{pro[t_s, t_e]}$ be the corresponding projected graph. The temporal k -core over $[t_s, t_e]$ is the

maximum subgraph $\mathcal{H} \subseteq \mathcal{G}_{pro[t_s, t_e]}$ such that each vertex in $\mathcal{H}$ has at least k neighbors.

Definition 3.2. (Coreness). Given a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and time interval $[t_s, t_e]$. The coreness of a vertex $v \in \mathcal{V}$ over $[t_s, t_e]$, denoted by $c_{[t_s, t_e]}(v)$, is the largest k value such that v belongs to a temporal k -core in $\mathcal{G}_{pro[t_s, t_e]}$.

Example 1. Take Figure 1 as an example. Let the time interval be $[3, 8]$. The vertices v_1 to v_8 form a temporal 2-core. All vertices v_1 through v_8 share a common coreness value of 2, i.e., $c_{[3, 8]}(v_1) = \dots = c_{[3, 8]}(v_8) = 2$.

Graph Neural Networks. GNNs [34] follow a neighborhood aggregation mechanism. This process can be expressed as:

$$h_v^{(l)} = M\left(h_v^{(l-1)}, \text{AGG}\left\{h_u^{(l-1)} : u \in N(v)\right\}\right) \quad (1)$$

Here, $h_v^{(l)}$ is the representation of vertex v in layer l , AGG is the aggregation function that combines messages from neighbors, and M is the message propagation function that updates the representation of the vertex using the aggregated representations and its own representation from the previous layer.

4 COCO Framework

To address the TICS problem, we present COCO, a novel cohesiveness-aware learning framework. The framework’s cohesiveness-aware nature stems from its integration of the classic k -core model with learning-based approaches. Compared to existing learning-based community search approaches, COCO is capable of identifying communities that demonstrate strong cohesiveness and can flexibly adapt to different graph structures across various queries.

Figure 2 shows COCO’s three-stage pipeline: pre-training, fine-tuning, and searching. (1) In pre-training, the HM-Index learns to predict vertex coreness over time, while the AT-GNN captures common temporal community patterns. (2) During fine-tuning, the HM-Index predicts coreness for query vertices V_Q and their neighbors in time interval T_Q . A sample extractor then creates positive and negative examples based on these coreness values and the query context, used to fine-tune the AT-GNN’s adapter module. (3) In the search phase, AT-GNN generates vertex embeddings. The progressive search module iteratively selects vertices most similar to V_Q , adding them to the community until constraints are met. Note that, due to space limitation, some proofs are moved to **technical report [1]**.

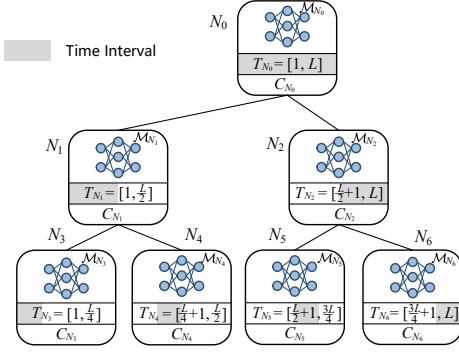


Figure 3: Architecture of HM-Index

4.1 Hierarchical Model Index

To integrate coreness with GNNs, we need to compute coreness for arbitrary query intervals T_Q , since different intervals may produce dramatically different graph topologies. Computing coreness dynamically for each query is accurate but costly, while precomputing coreness for all $O(t_{\max}^2)$ possible subintervals requires excessive storage and preprocessing time [44]. This presents a fundamental trade-off between efficiency and flexibility.

We propose a learning-based method that predicts coreness instead of computing it directly. Our approach divides the entire time span into smaller intervals and uses lightweight regression models to predict coreness within each. These models are organized hierarchically into a structure called the Hierarchical Model Index (HM-Index), illustrated in Figure 3. Each node N in the HM-Index has three components: (1) T_N : the time interval it covers; (2) M_N : a lightweight model that predicts vertex coreness for any subinterval of T_N ; (3) $C_N = \{c_{T_N}(v) \mid v \in \mathcal{V}\}$: the set of vertex coreness values over T_N , used by the parent node to improve prediction accuracy.

Time Interval Partition Rules. The HM-Index employs a recursive partitioning scheme: the root node spans the entire temporal range $[1, t_{\max}]$; each parent node divides its interval into p equal subintervals; partitioning terminates when leaf nodes' time interval length falls below the given threshold x . For example, in Figure 3, $p = 2$ and $x = \frac{L}{4}$. The HM-Index structure maintains two key properties: (1) *Strict Hierarchy*: for any child node N' of a node N , it satisfies that $T_{N'} \subset T_N$. (2) *Disjoint Siblings*: Nodes at the same depth have non-overlapping time intervals.

Model Structure. The regression model M_N in a node N is a lightweight Multilayer Perceptron (MLP). It consists of four components: (1) A linear transformation layer $\mathbf{W}_0 \in \mathbb{R}^{\dim_h \times 1}$ for single-valued features; (2) A timestamp feature processing layer $\mathbf{W}_1 \in \mathbb{R}^{\dim_h \times 2\lceil \frac{|T_N|}{p} \rceil}$; (3) A time interval feature processing layer $\mathbf{W}_2 \in \mathbb{R}^{\dim_h \times p}$; (4) A feature fusion layer $\mathbf{W}_f \in \mathbb{R}^{1 \times 3 \cdot \dim_h}$ that merges the three processed features into a unified representation. Here, $\dim_h$ represents the hidden dimension. The model processes different feature types separately and then fuses them, allowing specialized feature extraction while producing a more comprehensive representation.

Coreness Prediction Algorithm. The coreness prediction algorithm uses a multi-stage approach that traverses the HM-Index from the root, refining predictions at each level. We first describe

Algorithm 1: Coreness Prediction Algorithm

Input: A temporal graph $\mathcal{G}$, a vertex v , a query time interval $[t_s, t_e]$, HM-Index root node N_{root}

Output: Predicted coreness $\hat{c}$ for v over $[t_s, t_e]$

```

1  $N \leftarrow N_{root}; \hat{c} = 0;$ 
2 while True do
3    $\mathbf{s}_1 \leftarrow [c_t(v) \mid d_t(v)]_{t=t_s, t \notin [t'_s, t'_e]}$ ;
4    $\mathbf{s}_2 \leftarrow [C_{T_{N_i}} \mid d_{T_{N_i}}(v)]_{i=0}^{|N_c|-1}$ ;
5    $\hat{c} \leftarrow \mathcal{M}_N(\hat{c}, \mathbf{s}_1, \mathbf{s}_2);$  // Eq. (2)
6   for  $N' \in N.child$  do
7     if  $[t_s, t_e] \subseteq T_{N'}$  then
8        $N \leftarrow N';$  break;
9   if no child node contains  $[t_s, t_e]$  then break;
10 return  $\hat{c};$ 

```

the input to the model M_N , then present the full algorithm that coordinates this hierarchical prediction process.

Given an HM-Index node N , a vertex v , and a query time interval $[t_s, t_e] \subseteq T_N$, the interval may span multiple child nodes of N . Let $N_c = \{N_0, N_1, \dots, N_{|N_c|-1}\}$ be the ordered sequence of these child nodes, covering a combined interval $[t'_s, t'_e]$. Let N_p be the parent of N . The model M_N predicts the coreness of v as follows.

$$\hat{c}_{[t_s, t_e]}^N(v) = \mathcal{M}_N(\hat{c}, \mathbf{s}_1, \mathbf{s}_2) = \mathbf{W}_f(\mathbf{W}_0 \hat{c} \parallel \mathbf{W}_1 \mathbf{s}_1 \parallel \mathbf{W}_2 \mathbf{s}_2) \quad (2)$$

The model M_N takes three inputs to predict the coreness of a vertex v : $\hat{c}$, $\mathbf{s}_1$, and $\mathbf{s}_2$. $\hat{c}$ is the coreness value predicted by the parent node N_p , or 0 if no prediction exists. It helps refine the current prediction by leveraging information from the parent node. The vector $\mathbf{s}_1$ encodes the coreness and degree of v at timestamps outside a given interval $[t'_s, t'_e]$, capturing external temporal dynamics. The vector $\mathbf{s}_2$ aggregates v 's coreness and degree across all time intervals in the child nodes N_c , providing a summary of its structural properties over time.

Based on the coreness prediction of a single HM-Index node, the complete coreness prediction algorithm based on the HM-Index is outlined in Algorithm 1. It starts by setting the root node N and initializing the predicted coreness $\hat{c}$ to 0 (line 1). Then, Algorithm 1 traverses the HM-Index as follows. For the currently visited HM-Index node N , two feature vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ are computed (lines 3–4), based on which the coreness $\hat{c}$ is updated (line 5). Next, Algorithm 1 checks if there exists an N 's child node N' that fully covers the query time interval $[t_s, t_e]$ (lines 6–8). If Yes, Algorithm 1 visits N' to update the coreness. Otherwise, Algorithm 1 terminates. By dynamically retrieving the index and using rich feature vectors, Algorithm 1 predicts coreness with high accuracy and efficiency.

Theoretical Error Analysis. We model HM-Index's hierarchical prediction as a sequential refinement process. During the root-to-leaf traversal in Algorithm 1, each node uses increasingly localized temporal features to correct its parent's prediction. The following lemma proves that this top-down residual learning tightly bounds the final prediction error.

LEMMA 4.1. Assume each model M_{N_j} at depth j in Algorithm 1 satisfies the contraction property $\epsilon_j \leq \beta \epsilon_{j-1} + \eta$, where $\beta \in [0, 1)$ is the contraction rate, $\eta \geq 0$ is the irreducible local error, and $\epsilon_j =$

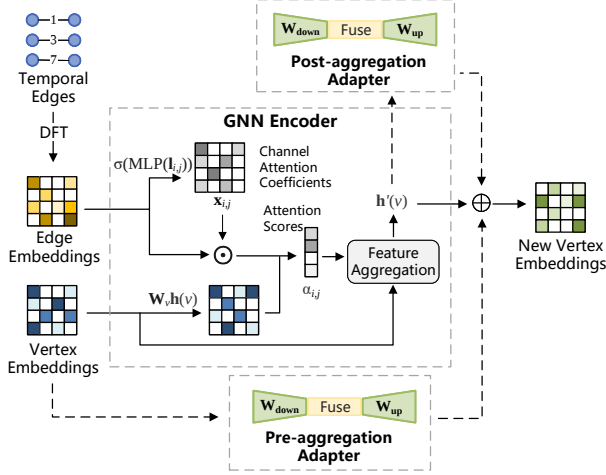


Figure 4: Architecture of AT-GNN

$|\hat{c}^{(j)} - c_{[t_s, t_e]}^*(v)|$. The final prediction error after traversing h levels is strictly bounded by $\epsilon_h \leq \beta^h \epsilon_0 + \frac{\eta(1-\beta^h)}{1-\beta}$.

PROOF. HM-Index operates as a coarse-to-fine residual chain. At depth j , $\mathcal{M}_{N_j}$ treats the parent prediction $\hat{c}^{(j-1)}$ as a structural prior, utilizing local temporal features to learn a residual correction. Since deeper nodes cover progressively narrower time intervals, the local estimation task simplifies, naturally inducing the contraction rate $\beta < 1$ alongside a bounded local approximation error η .

We prove $\epsilon_j \leq \beta^j \epsilon_0 + \eta \sum_{k=0}^{j-1} \beta^k$ by induction. The base case $j = 0$ is trivial. Assuming this holds for depth j , the contraction property yields $\epsilon_{j+1} \leq \beta \epsilon_j + \eta \leq \beta(\beta^j \epsilon_0 + \eta \sum_{k=0}^{j-1} \beta^k) + \eta = \beta^{j+1} \epsilon_0 + \eta \sum_{k=0}^j \beta^k$, completing the induction.

Evaluating at the final depth h and computing the geometric series yields $\epsilon_h \leq \beta^h \epsilon_0 + \frac{\eta(1-\beta^h)}{1-\beta}$. Thus, the initial propagated error ϵ_0 decays exponentially with depth, while the cumulative local errors strictly converge to $\frac{\eta}{1-\beta}$, preventing error explosion and guaranteeing robust prediction accuracy. $\square$

Complexity Analysis. Let $t_{\max}$ be the maximum timestamp of the graph, and x be the maximum length of the time interval for the leaf nodes in the HM-Index. The number of leaf nodes in HM-Index is bounded by $O(\frac{t_{\max}}{x})$. Since HM-Index is a balanced tree, the total number of nodes is also bounded by $O(\frac{t_{\max}}{x})$. The height of HM-Index is $h = \log_p \frac{t_{\max}}{x}$. In the worst case, Algorithm 1 traverses the HM-Index from the root to the leaf node. Hence, the time complexity of Algorithm 1 is bounded by $O(\log \frac{t_{\max}}{x})$. Each node contains a model and a set of corenesses for vertices over the corresponding time interval. Therefore, the space required for a single node is bounded by $O(|\mathcal{M}| + |\mathcal{V}|)$. Consequently, the overall space complexity of HM-Index is bounded by $O(\frac{t_{\max}}{x} \cdot (|\mathcal{M}| + |\mathcal{V}|))$.

4.2 Adaptive Temporal GNN

The Adaptive Temporal Graph Neural Network (AT-GNN) is designed to capture both structural and temporal features of the temporal graph while enabling efficient fine-tuning for various TICS queries.

Figure 4 shows the architecture of one layer in the AT-GNN. The AT-GNN contains K layers. The model has two main components: the encoder and the adapter. Each layer includes two adapters, namely the *Pre-aggregation Adapter* and the *Post-aggregation Adapter*. The vertex embedding $h^{(l)}(v)$ in the l -th layer of the AT-GNN is computed as follows.

$$h^{(l)}(v) = \text{ENC}(h^{(l-1)}(v)) + \text{ADP}_{\text{pre}}(h^{(l-1)}(v)) + \text{ADP}_{\text{post}}(\text{ENC}(h^{(l-1)}(v))) \quad (3)$$

Here, $\text{ENC}(\cdot)$ is the encoder, $\text{ADP}_{\text{pre}}(\cdot)$ and $\text{ADP}_{\text{post}}(\cdot)$ are pre-aggregation and post-aggregation adapters, respectively. Next, we introduce them in detail.

Encoder. For a vertex v , the initial feature $h^{(0)}(v) \in \mathbb{R}^{\dim_v}$ is defined as:

$$h^{(0)}(v) = f_v([q(v) || s_c(v)]), \quad (4)$$

where $q(v)$ indicates whether v is a query vertex (1 if $v \in V_Q$, else 0), and $s_c(v) = [c_t(v)]_{t=t_s}^{t_e}$ is the temporal coreness sequence. The function $f_v(\cdot)$ adjusts dimension via linear interpolation (for short sequences) or average pooling (for long ones). To capture temporal interaction patterns, vertex and edge features are transformed into the frequency domain using the Discrete Fourier Transform (DFT). For an edge (v, v') , the initial embedding $\mathbf{l}_{v,v'} \in \mathbb{R}^{\dim_e}$ is

$$\mathbf{l}_{v,v'} = [\text{DFT}(f_e([a_t(v, v')]_{t=t_s}^{t_e}))]_{0:\dim_e}. \quad (5)$$

The edge presence sequence $[a_t(v, v')]_{t=t_s}^{t_e}$ indicates whether edge (v, v') exists at time t , where $a_t(v, v') = 1$ if connected and 0 otherwise. The function $f_e(\cdot)$ performs dimension transformation, similar to $f_v(\cdot)$.

LEMMA 4.2. Let $\mathbf{a}, \mathbf{b} \in \{0, 1\}^T$ be two edge presence sequences with Hamming distance $\|\mathbf{a} - \mathbf{b}\|_0 = m$, and let $\phi_d(\mathbf{a}) = [\hat{a}_k]_{k=0}^{d-1}$ denote the truncated DFT amplitude embedding defined in Eq. (5) with $d \leq T$. Then $\|\phi_d(\mathbf{a}) - \phi_d(\mathbf{b})\|_2 \leq \sqrt{m}$.

Lemma 4.2 guarantees that edges with similar temporal patterns receive similar embeddings, ensuring robustness of the attention scores in Eq. (7) to small temporal perturbations. After applying DFT, the first $\dim_e$ elements of the amplitude vector form the edge embedding. To strengthen temporal modeling, a channel attention mechanism refines the embeddings:

$$\mathbf{x}_{v,v'} = \sigma(\text{MLP}(\mathbf{l}_{v,v'})), \quad \mathbf{l}'_{v,v'} = \mathbf{l}_{v,v'} \odot \mathbf{x}_{v,v'}. \quad (6)$$

For vertex aggregation, the encoder calculates attention scores $b_{v,v'}$ by combining vertex and edge features:

$$b_{v,v'} = \text{LeakyReLU}(\mathbf{a}^T \cdot [\mathbf{W}_v \mathbf{h}^{(l-1)}(v) || \mathbf{W}_v \mathbf{h}^{(l-1)}(v') || \mathbf{l}'_{v,v'}]). \quad (7)$$

Here, $\mathbf{a}$ is a learnable vector and $\mathbf{W}_v$ projects vertex features. Dynamic edge features $\mathbf{l}'_{v,v'}$ capture temporal interactions. Then, attention scores are computed and normalized, then used to aggregate neighboring features to update $h^{(l)}(v)$, as in Eq. (8).

$$\alpha_{v,v'} = \text{softmax}_{v'}(b_{v,v'}) = \frac{\exp(b_{v,v'})}{\sum_{v'' \in \mathcal{N}(v) \cup \{v\}} \exp(b_{v,v''})}, \quad (8)$$

$$\text{ENC}(h^{(l-1)}(v)) = \sum_{v' \in \mathcal{N}(v) \cup \{v\}} \alpha_{v,v'} \cdot h^{(l-1)}(v')$$

Adapter. COCO is designed to handle TICS queries under varying graph structures. Fully retraining or fine-tuning the encoder for

Algorithm 2: Forward Propagation of AT-GNN

Input: A temporal graph $\mathcal{G}$, a set of query vertices V_Q , a query time interval T_Q

Output: The embeddings of $\mathcal{V}$

```

1  $\{\mathbf{h}^{(0)}(v) \leftarrow f_v([q(v)||s_c(v)]) \mid v \in \mathcal{V}\}$  // Eq. (4)
2  $\{I_{v,v'} \leftarrow [|\text{DFT}(f_e([a_t(v,v')])_{t=t_s}^{t_e})|_{0:\text{dim}_e}] \mid e = (v,v'), t \in \mathcal{E}, t \in T_Q\}$  // Eq. (5)
3 for  $l = 1$  to  $K$  do
4   for  $v \in \mathcal{V}$  do
5      $\mathbf{h}_{\text{pre}}(v) \leftarrow \text{ADP}(\mathbf{h}^{(l-1)}(v))$ ; // Pre-aggregation adapter
6     // encoder Eq. (6) to (8)
7     for  $v' \in \{v\} \cup N_{T_Q}(v)$  do
8        $\mathbf{x}_{v,v'} \leftarrow \sigma(\text{MLP}(I_{v,v'}))$ ,  $\mathbf{l}'_{v,v'} \leftarrow \mathbf{l}_{v,v'} \odot \mathbf{x}_{v,v'}$ ;
9        $b_{v,v'} = \text{LeakyReLU}(\mathbf{a}^T \cdot [\mathbf{W}_v \mathbf{h}^{(l-1)}(v) || \mathbf{W}_{v'} \mathbf{h}^{(l-1)}(v') || \mathbf{l}'_{v,v'}])$ ;
10       $\alpha_{v,v'} = \text{softmax}_{v'}(b_{v,v'})$ ;
11       $\mathbf{h}'(v) \leftarrow \sum_{v' \in N(v) \cup \{v\}} \alpha_{v,v'} \cdot \mathbf{h}^{(l-1)}(v')$ ;
12       $\mathbf{h}_{\text{post}}(v) \leftarrow \text{ADP}(\mathbf{h}'(v))$ ; // Post-aggregation adapter
13       $\mathbf{h}^{(l)} \leftarrow \mathbf{h}_{\text{pre}}(v) + \mathbf{h}'(v) + \mathbf{h}_{\text{post}}(v)$ ;
14 return  $\{\mathbf{h}^{(l)}(v) \mid v \in \mathcal{V}\}$ ;

Procedure  $\text{ADP}(\mathbf{h}_{\text{in}}(v))$ 
15  $\mathbf{s}_Q \leftarrow [\bar{c}(V_Q) || \bar{c}(N(V_Q)) || \frac{t_s}{t_{\max}} || \frac{|T_Q|}{t_{\max}}]$ ;
16  $\tilde{\mathbf{h}}(v) \leftarrow [\mathbf{W}_{\text{down}_1} \mathbf{h}_{\text{in}}(v) || \mathbf{W}_{\text{down}_2} \mathbf{s}_Q]$ ; // Eq. (9)
17  $\mathbf{q} \leftarrow \mathbf{W}_q \tilde{\mathbf{h}}(v)$ ,  $\mathbf{k} \leftarrow \mathbf{W}_k \tilde{\mathbf{h}}(v)$ ,  $\mathbf{v} \leftarrow \mathbf{W}_v \tilde{\mathbf{h}}(v)$ ; // Eq. (9)
18  $\alpha \leftarrow \text{softmax}(\frac{\mathbf{q}^T \mathbf{k}}{\sqrt{\text{dim}_{\text{head}}}})$ ; // Eq. (10)
19 return  $\text{ReLU}(\mathbf{W}_{\text{up}}(\alpha \cdot \mathbf{v}))$ ; // Eq. (10)

```

different queries would be prohibitively expensive. To address this issue, we design a query context-aware adapter. This lightweight trainable module is inserted into each encoder layer, both before and after the aggregation step. This dual-adapter architecture improves generalization by enabling controlled feature adaptation while preserving the original graph representations.

Each adapter includes three parts: dimension reduction, feature fusion, and dimension expansion. In the dimension reduction stage, two inputs are processed. First, vertex embeddings $\mathbf{h}_{\text{in}}(v)$ are linearly projected to a lower dimension using $\mathbf{W}_{\text{down}_1}$. Second, query context features are extracted: average coreness of query vertices and their neighbors ($\bar{c}(V_Q)$ and $\bar{c}(N(V_Q))$), and normalized time features $\frac{t_s}{t_{\max}}$ and $\frac{|T_Q|}{t_{\max}}$. These features are combined into a vector $\mathbf{s}_Q$, then projected via $\mathbf{W}_{\text{down}_2}$. In the feature fusion module, a self-attention mechanism combines $\mathbf{h}_{\text{ADP}}(v)$ and $\mathbf{s}_{\text{ADP}}$. After concatenation, three learnable matrices $\mathbf{W}_q$, $\mathbf{W}_k$, and $\mathbf{W}_v$ project the input into query, key, and value vectors, as shown in Eq. (9).

$$\begin{aligned} \tilde{\mathbf{h}}(v) &= [\mathbf{W}_{\text{down}_1} \mathbf{h}_{\text{in}}(v) || \mathbf{W}_{\text{down}_2} \mathbf{s}_Q], \\ \mathbf{q} &= \mathbf{W}_q \tilde{\mathbf{h}}(v), \quad \mathbf{k} = \mathbf{W}_k \tilde{\mathbf{h}}(v), \quad \mathbf{v} = \mathbf{W}_v \tilde{\mathbf{h}}(v) \end{aligned} \quad (9)$$

In the final dimension expansion stage, the output is further transformed by a learnable matrix $\mathbf{W}_{\text{up}}$. The complete adapter output for input $\mathbf{h}_{\text{in}}(v)$ is given in Eq. (10):

$$\alpha = \text{softmax}\left(\frac{\mathbf{q}^T \mathbf{k}}{\sqrt{\text{dim}_{\text{head}}}}\right), \quad \text{ADP}(\mathbf{h}_{\text{in}}(v)) = \text{ReLU}(\mathbf{W}_{\text{up}}(\alpha \cdot \mathbf{v})) \quad (10)$$

Theoretically, this reduce-then-expand design guarantees parameter efficiency without compromising the model's capacity for query-specific adjustments.

LEMMA 4.3. *For a fixed query context $\mathbf{s}_Q$, the adapter induces a residual update whose Jacobian has rank at most $\xi = \min\{\text{dim}_{\text{ADP}}, \text{dim}_{\text{head}}\}$, and can exactly realize any query-specific affine correction of rank at most ξ .*

Lemma 4.3 shows that the down-projection/up-projection bottleneck introduces a *query-conditioned low-rank* residual to the encoder representation. When the desired query-specific correction has intrinsic rank no greater than the bottleneck dimension r , the adapter can recover it exactly while using far fewer parameters than full fine-tuning. This makes the design particularly suited to TICS queries, where adaptation is typically driven by a small number of structural and temporal factors.

Forward Propagation. As shown in Algorithm 2, forward propagation of AT-GNN begins by computing initial embeddings for vertices and edges (lines 1–2). Then, the model performs K layers of message passing. Each layer includes a pre-aggregation adapter, an encoder, and a post-aggregation adapter. In the pre-aggregation adapter, the input embedding $\mathbf{h}^{(l-1)}(v)$ is transformed into an intermediate representation $\mathbf{h}_{\text{pre}}(v)$ using the ADP procedure (line 5). ADP improves embeddings by adding query-specific context. It builds a context vector $\mathbf{s}_Q$ from the query vertex set V_Q and its neighbors, applies self-attention, and outputs a refined representation (lines 14–18). Next, the encoder aggregates neighbor information. For each neighbor v' of v , attention scores $\alpha_{v,v'}$ are calculated using channel attention, and used to combine neighbor embeddings (lines 6–10). The result, $\mathbf{h}'(v)$, is further processed by the post-aggregation adapter via ADP (line 11). Finally, the output combines features from all three components (line 12).

5 COCO Workflow

In this section, we present the workflow of the COCO, which includes pre-training, fine-tuning, and searching phases.

5.1 Pre-training Phase

In the pre-training phase, COCO builds the HM-Index and trains the AT-GNN. Next, we provide a detailed explanation of training for these two components.

Construction for HM-Index. The HM-Index construction algorithm constructs a hierarchical tree structure to predict vertex coreness in temporal graphs. Using a top-down strategy, the algorithm recursively divides time intervals into smaller segments. For each resulting time interval, it creates a corresponding tree node and trains a specialized prediction model.

As outlined in Algorithm 3, the procedure starts by initializing a queue L containing the root node. The root node covers the entire time interval $\mathcal{T}$ (line 1). For each node N from the queue, the algorithm first creates a model (line 4) and then randomly samples batches of vertices (line 5). For each vertex v in the batch, it selects a

Algorithm 3: HM-Index Construction

Input: A temporal graph $\mathcal{G}$, a partition coefficient p , a length threshold x , a learning rate φ , a number of epochs λ , a batch size n_b

Output: Root node of HM-Index

```

1 Initialize queue  $L$  with a node  $N_{root}$  with entire time interval  $\mathcal{T}$ ;
2 while  $L \neq \emptyset$  do
3    $N \leftarrow L.dequeue()$ ; initialize a model  $\mathcal{M}_N$  for  $N$ ;
4   Sample batches  $\{V_1, V_2, \dots, V_n\}$  where each  $|V_{b_i}| = n_b$ ;
5   for  $epoch = 1$  to  $\lambda$  do
6     for each batch  $V_i \in \{V_1, V_2, \dots, V_n\}$  do
7        $\mathcal{L}_c \leftarrow 0$ ;
8       for  $v \in V_i$  do
9          $[t_s, t_e] \leftarrow$  randomly sample subinterval from  $T_N$ ;
10         $c(v)_{[t_s, t_e]} \leftarrow$  compute the coreness of  $v$ ;
11         $\hat{c}_{[t_s, t_e]}(v) \leftarrow$  predict using Algorithm 1;
12         $\mathcal{L}_c \leftarrow \mathcal{L}_c + \frac{1}{n_b} (\hat{c}(v) - c(v))^2$ ;
13       $\theta(\mathcal{M}_N) \leftarrow \theta(\mathcal{M}_N) - \varphi \cdot \nabla_{\theta} \mathcal{L}_c$ ;
14   if  $|T_N| > x$  then
15     Create  $p$  child nodes with equal-length subintervals;
16     Enqueue child nodes into  $L$ ;
17 return  $N_{root}$ ;

```

random subinterval of T_N (line 9). The coreness of v over this subinterval is computed using the core decomposition method [15] and serves as the ground-truth label (line 10). Meanwhile, Algorithm 1 generates the predicted coreness of v (line 11). The training objective is defined as the mean squared error between the predicted and actual values.

$$\mathcal{L}_c = \frac{1}{n_b} \sum_{i=1}^{n_b} (\hat{c}(v_i) - c(v_i))^2 \quad (11)$$

This loss is calculated for each batch (line 12), and the parameters of the model $\mathcal{M}_N$ are updated accordingly (line 13). If the current node's time interval has a length greater than or equal to x , it is divided into p subintervals (line 15), and child nodes corresponding to these new intervals are enqueued (line 16). This iterative process continues until all eligible nodes are processed, after which the HM-Index is constructed.

Training for AT-GNN. Most learning-based community search methods depend on ground-truth communities. However, this approach does not work for temporal graphs because there are no ground-truth communities available for existing temporal graphs [21]. To address this, we propose a contrastive learning method to train AT-GNN in an unsupervised manner. We design two loss functions for training AT-GNN: the cohesive loss and the temporal link loss.

The cohesive loss function is designed to bring the embeddings of potentially related vertices (likely belonging to the same community) closer together in the latent space, while simultaneously pushing unrelated vertices farther apart. To achieve this, we define a cohesive score function that evaluates how strongly a vertex is connected to the query vertex V_Q within the time interval T_Q .

$$CoS(v) = c(v) \cdot \frac{|\{e = (v, u, t) | u \in N(v), t \in T_Q\}|}{|N_{T_Q}(v)|} \quad (12)$$

Sampling Rule 1. To achieve contrastive learning for AT-GNN, positive and negative samples are generated for the cohesive loss function.

(1) Positive Samples V_{pos} : V_{pos} is built by iteratively selecting the neighbor v' with the highest $CoS(v')$, until the number of positive samples reaches the preset threshold $|V_{smp}|$.

(2) Negative Samples V_{neg} : V_{neg} is formed by randomly selecting $|V_{smp}|$ vertices from $\{\mathcal{V} - V_{pos}\}$.

Once the positive and negative samples are identified, we model the cohesive loss using margin triplet loss [30]. It minimizes the distance between the query vertex and positive samples while enforcing a margin between the query and negative samples.

$$\mathcal{L}_{Coh} = \frac{1}{|V_{pos}| \cdot |V_{neg}|} \cdot \sum_{u \in V_{pos}} \sum_{v \in V_{neg}} \max(0, dis(u, V_Q) - dis(v, V_Q) + \epsilon) \quad (13)$$

Here, $dis(\cdot)$ computes the average L2 distance between a vertex and the query vertex. For example, $dis(v, V_Q) = \frac{1}{|V_Q|} \cdot \sum_{v' \in V_Q} \|h(v) - h(v')\|$. ϵ is the margin value.

A vertex may have a different number of connections with its neighbors. The idea behind the temporal link loss is to enhance the similarity of vertices with frequent connections while distinguishing weakly linked vertices. To measure the strength of the connection between two vertices, we define a function of link strength.

$$E(v, u) = \{e = (v, u, t) | e \in \mathcal{E}, t \in T_Q\},$$

$$LinkS(v, u) = \frac{|E(v, u)|}{\max_{(v, u, t) \in E(v, u)} t - \min_{(v, u, t) \in E(v, u)} t + 1} \quad (14)$$

Sampling Rule 2. The samples for the temporal link loss are triples, denoted as V_{temp} . Each triple $(v, v_p, v_n) \in V_{temp}$ consists of an anchor vertex, a positive vertex, and a negative vertex.

(1) Anchor Vertex v : v is randomly selected from $\mathcal{V}$.

(2) Positive Vertex v_p : Among the neighbors of v , the positive vertex is $v_p = \arg \max_{v' \in N_{T_Q}(v)} LinkS(v, v')$.

(3) Negative Vertex v_n : Among the neighbors of v , the negative vertex is $v_n = \arg \min_{v' \in N_{T_Q}(v)} LinkS(v, v')$.

Based on the sampled triples, the temporal link loss is defined as Eq.(15). Finally, we combine the cohesive loss and the temporal link loss as Eq.(16).

$$\mathcal{L}_{Temp} = \frac{1}{|V_{temp}|} \cdot \sum_{(v_a, v_p, v_n) \in V_{temp}} \max(0, dis(v_a, v_p) - dis(v_a, v_n) + \epsilon) \quad (15)$$

$$\mathcal{L} = \mathcal{L}_{Coh} + \delta \mathcal{L}_{Temp} \quad (16)$$

The overall training process of AT-GNN is presented in Algorithm 4. The algorithm starts by generating several query batches (line 1). Each query has a different query vertex and a different time interval. This allows AT-GNN to learn community information across various time intervals. For each query, the algorithm samples data for the cohesive loss and computes the loss (lines 7-8). It then samples data for the temporal link loss and computes the corresponding loss (lines 9-10). These two losses are combined (line 11). After processing each batch, the parameters of AT-GNN are updated (line 12).

Algorithm 4: AT-GNN Pre-training

Input: A temporal graph $\mathcal{G}$, a learning rate φ , a number of epochs λ , a batch size n_b , a initialized AT-GNN model $\mathcal{M}_{AT}$

Output: The trained model $\mathcal{M}_{AT}$

- 1 Generate queries batches $\{Q_1, Q_2, \dots, Q_n\}$ where Q_i has n_b queries;
- 2 **for** $epoch = 1$ **to** λ **do**
- 3 **for** each batch $Q_i \in \{Q_1, Q_2, \dots, Q_n\}$ **do**
- 4 $\mathcal{L} \leftarrow 0$;
- 5 **for** each query $(V_Q, T_Q) \in Q_i$ **do**
- 6 $\{h^{(l)}(v) \mid v \in \mathcal{V}\} \leftarrow \text{Algorithm 2}$;
- 7 $V_{pos}, V_{neg} \leftarrow \text{sample by rule 1}$;
- 8 $\mathcal{L}_{Coh} \leftarrow \frac{1}{n_b} \cdot \frac{1}{|V_{pos}| \cdot |V_{neg}|} \sum_{u \in V_{pos}} \sum_{v \in V_{neg}} \max(0, dis(u, V_Q) - dis(v, V_Q) + \epsilon)$;
- 9 $V_{temp} \leftarrow \text{sample by rule 2}$;
- 10 $\mathcal{L}_{Temp} \leftarrow \frac{1}{n_b} \cdot \frac{1}{|V_{temp}|} \sum_{(v_a, v_p, v_n) \in V_{temp}} \max(0, dis(v_a, v_p) - dis(v_a, v_n) + \epsilon)$;
- 11 $\mathcal{L} \leftarrow \mathcal{L}_{Coh} + \delta \mathcal{L}_{Temp}$;
- 12 $\theta(\mathcal{M}_{AT}) \leftarrow \theta(\mathcal{M}_{AT}) - \varphi \cdot \nabla_{\theta} \mathcal{L}$;
- 13 **return** $\mathcal{M}_{AT}$;

5.2 Fine-tuning Phase

In the fine-tuning phase, the adapter module in AT-GNN plays a key role. When a query arrives, COCO avoids heavy query overhead by fine-tuning only the parameters in the adapter module. The process of fine-tuning is shown in Algorithm 5.

First, COCO gets the coreness of all vertices using HM-Index via Algorithm 2 (line 1). Then it samples vertices based on Rule 1 and Rule 2 from Section 5.1 (lines 2). For each fine-tuning step, COCO generates the vertex embeddings using AT-GNN (line 4). Next, sampled vertices are used to compute the loss (line 5). The key difference between pre-training and fine-tuning lies in parameter updates. During pre-training, all parameters in AT-GNN are updated. However, during fine-tuning, the parameters in the GNN encoder are frozen. Only the parameters in the adapter module are updated based on the computed loss (line 6).

This approach offers several benefits. First, freezing the GNN encoder reduces computational costs significantly. Since the encoder parameters remain unchanged, the model avoids redundant updates, making the fine-tuning process more efficient. Second, fine-tuning only the adapter module allows the model to adapt quickly to new queries while retaining the general knowledge learned during pre-training. This ensures that the model remains robust and accurate even with limited query-specific data.

5.3 Searching Phase

The searching phase identifies the final community for TICS. COCO progressively expands the result set by adding vertices whose embeddings are similar to those of the query vertex. We introduce a novel adaptive search algorithm that dynamically adjusts the termination threshold.

We achieve this dynamic adjustment through an adaptive update function (Eq.(17)). Based on small-world theory [16, 25], community members exhibit strong locality, so we first extract an r -hop

Algorithm 5: COCO Fine Tuning & Searching

Input: A temporal graph $\mathcal{G}$, a set of query vertices V_Q , a query time interval T_Q , a learning rate φ , a number of fine tuning λ , the AT-GNN $\mathcal{M}_{AT}$ model, adapter modules $\mathcal{M}_{ADP}$, HM-Index root node N_{root}

Output: The resulting community C

// Fine-tuning Phase

- 1 $\{\hat{c}(v) \mid v \in \mathcal{V}\} \leftarrow \text{predict corenesses using } N_{root} \text{ via Algorithm 1}$;
- 2 $V_{pos}, V_{neg}, V_{temp} \leftarrow \text{sample by rules 1 and 2}$;
- 3 **for** $epoch = 1$ **to** λ **do**
- 4 $\mathcal{L} \leftarrow 0$; $\{h^{(l)}(v) \mid v \in \mathcal{V}\} \leftarrow \text{infer with } \mathcal{M}_{AT} \text{ via Algorithm 2}$;
- 5 $\mathcal{L} \leftarrow \text{compute the loss function based on } V_{pos}, V_{neg}, V_{temp}$;
- 6 $\theta(\mathcal{M}_{ADP}) \leftarrow \theta(\mathcal{M}_{ADP}) - \varphi \cdot \nabla_{\theta} \mathcal{L}$;
- 7 // Searching Phase
- 7 $C \leftarrow V_Q$; $\{h^{(l)}(v) \mid v \in \mathcal{V}\} \leftarrow \text{infer with } \mathcal{M}_{AT} \text{ via Algorithm 2}$;
- 8 $V_r \leftarrow r\text{-hop neighbors of } V_Q$;
- 9 $\tau \leftarrow \frac{\sum_{v \in V_r} dis(v, V_Q)}{|V_r - V_Q|}$; $min_dis \leftarrow 0$;
- 10 **while** $min_dis < \tau$ **do**
- 11 $C \leftarrow C \cup \arg \min_{v \in N_{T_Q}(C) \cap V_r} dis(v, V_Q)$;
- 12 $min_dis \leftarrow \min_{v \in N_{T_Q}(C) \cap V_r} dis(v, V_Q)$;
- 13 $\tau \leftarrow \cos(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|}) \cdot \tau + (1 - \cos(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|})) \cdot \frac{\sum_{v' \in C} dis(v', T_Q)}{|C|}$;
- 14 **return** C ;

subgraph around the query vertex, denoted as V_r . Initially, the threshold τ is set to the average embedding distance between vertices in the r -hop subgraph $G_r = (V_r, E_r)$ and the query vertex V_Q : $\tau = \frac{\sum_{v \in V_r} dis(v, V_Q)}{|V_r - V_Q|}$. During each iteration, COCO selects the vertex with the minimum distance and adds it to the result set C . For the n -th iteration, when a new vertex is added to C , the threshold τ is updated according to the following function:

$$\tau_n = \rho_n \cdot \tau_{n-1} + (1 - \rho_n) \cdot \frac{\sum_{v \in C} dis(v, V_Q)}{|C|}, \quad \rho_n = \cos\left(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|}\right) \quad (17)$$

Here, $\rho_n = \cos\left(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|}\right)$ acts as a smoothing factor that progressively adjusts τ at each step. The search process terminates when the minimum distance among remaining candidate vertices surpasses the current τ value. This mechanism possesses the following important property.

LEMMA 5.1. *The smoothing factor $\rho_n = \cos\left(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|}\right)$ decreases monotonically with community size $|C|$ and satisfies $\lim_{|C| \rightarrow |V_r|} \rho_n = 0$, guaranteeing that τ converges to $\frac{\sum_{v \in C} dis(v, V_Q)}{|C|}$.*

PROOF. As $|C|$ increases from $|V_Q|$ to $|V_r|$, $\frac{\pi}{2} \cdot \frac{|C|}{|V_r|}$ grows from $\frac{\pi}{2} \cdot \frac{|V_Q|}{|V_r|}$ to $\frac{\pi}{2}$. Within this interval, the cosine function is strictly decreasing, which means ρ_n strictly decreases and approaches 0. In the update equation, when ρ_n approaches 0, the term $\rho_n \cdot \tau_{n-1}$ becomes negligible while $(1 - \rho_n)$ tends toward 1. Consequently, τ_n becomes increasingly determined by the current average distance $\frac{\sum_{v \in C} dis(v, V_Q)}{|C|}$. Since vertices are added in order of increasing distance, this average distance is non-decreasing and eventually stabilizes, causing τ to converge to this stabilized average distance. $\square$

Our adaptive threshold offers two key advantages. First, the dynamic adjustment of τ automatically determines the search scope, eliminating the need for fixed boundaries. Second, leveraging the slope characteristics of the cosine function, the initial flat gradient maintains threshold stability to prevent premature termination, while the subsequent steep descent accelerates convergence to curb excessive expansion.

Overall, the search process is presented in lines 7-14 of Algorithm 5. First, the result set is initialized with the query vertex. Embeddings of all vertices are obtained through the inference of AT-GNN (line 7). Next, the r -hop subgraph of the query vertex is extracted (line 8). The search threshold is then set to the average distance within the subgraph (line 9). The algorithm adds the neighboring vertex with the smallest distance into the result set (line 11). After that, the minimal distance is updated (line 12), and the threshold is adjusted accordingly (line 13). Once the minimal distance exceeds the threshold, the search process stops.

5.4 Time Complexity Analysis

Pre-training Phase The time complexity contains two components. First, the HM-Index tree construction involves building a tree structure with depth $h = \log_p \frac{t_{\max}}{x}$, where $t_{\max}$ is the maximal timestamp, x the leaf interval length, and p the branching factor. The core decomposition process requires $O(|V| + |E|)$ time in total across all timestamps. The HM-Index structure contains $O(\frac{t_{\max}}{x})$ nodes. For each node, the training process requires $O(|V| \cdot \log \frac{t_{\max}}{x})$ time per epoch. This leads to an overall time complexity of $O(|E| + \lambda \cdot \frac{t_{\max}}{x} \cdot |V| \cdot \log \frac{t_{\max}}{x})$. Second, the AT-GNN processes K GNN layers with feature dimension d . Each layer requires $O(|V| + |E|)$ time. Training on Q queries, this results in a time complexity of $O(\lambda \cdot Q \cdot K \cdot (|V| + |E|))$. Overall, the total time complexity is $O(\lambda \cdot \frac{t_{\max}}{x} \cdot |V| \cdot \log \frac{t_{\max}}{x} + \lambda \cdot Q \cdot K \cdot (|V| + |E|))$.

Fine-tuning Phase. The computational cost in this phase is dominated by updating the parameters in the adapters, while the encoder parameters remain frozen. Consequently, fine-tuning the adapters requires $O(\lambda \cdot Q \cdot K \cdot |V|)$ time.

Searching Phase. The process begins with predicting vertex core-nesses in the r -hop subgraph using HM-Index, which requires $O(|V_r| \cdot \log \frac{t_{\max}}{x})$ time. Next, vertex embeddings are generated by AT-GNN and subsequently sorted, taking $O(K \cdot (|V_r| + |E_r|) + |V_r| \cdot \log |V_r|)$ time. Finally, the search process completes in $O(|V_r|)$ iterations. Therefore, the overall time complexity is $O(K \cdot (|V_r| + |E_r|) + |V_r| \cdot (\log |V_r| + \log \frac{t_{\max}}{x}))$.

6 Performance Evaluation

This section evaluates the performance of COCO. Our experiments utilize eight real-world temporal networks, whose statistical properties are summarized in Table 1. All experiments were conducted on an Ubuntu server, equipped with an Intel i9-10900K CPU, 128GB of memory, and an Nvidia RTX 3090 GPU.

Competitors. We compare COCO with seven competitors.

- *QTCS* [21]. A non-learning method that uses time-constrained personalized pagerank to find a β -temporal proximity core containing the query vertex.

Table 1: Statistics of networks

Dataset	Abbr.	$ V $	$ E $	$t_{\max}$
CollegeMsg	CM	1.8K	59.8K	193
MathOverflow	MO	24.8K	506.5K	2351
Superuser	SU	194.1K	1.4M	2774
WikiTalk	WT	1.1M	7.8M	2321
Youtube	YT	3.2M	9.3M	226
Flickr	FL	2.3M	33.1M	394
StackOverflow	SO	2.6M	63.5M	2774
Wikipedia	WP	6.9M	129.9M	298

- *Temporal k -core* [43]. A non-learning method that identifies a k -core in the projected graph of the given time interval. We employ an indexing-based method [44] to speed up its search process.
- *TransZero* [35]. An unsupervised learning framework for community search using graph transformer.
- *CSFormer* [39]. An unsupervised learning-based approach for static graphs that adds vertices whose predicted core-nesses are higher than that of the query vertices into the community.
- *IACS* [6]. An inductive learning-based method for community search across different static graphs.
- *CS-TGN* [11]. A supervised learning-based method designed for community search over temporal graphs.
- *UTCS* [45]. An unsupervised method for temporal graphs that pre-trains on temporal dynamics and subgraph knowledge to identify communities using a proximity-based scoring mechanism.

Metrics. Given the absence of ground-truth datasets for temporal graphs [21], we adopt the established evaluation framework from prior work [4, 21, 31], utilizing two widely recognized metrics: Temporal Density (TD) and Temporal Conductance (TC). For a query interval $[t_s, t_e]$ and retrieved community C , TD and TC are formally defined as follows:

$$TD(C) = \frac{2 \cdot |\{(v, u, t) \in \mathcal{E}_{[t_s, t_e]} | v, u \in C\}|}{|C| \cdot (|C| - 1) \cdot (t_e - t_s + 1)} \quad (18)$$

$$TC(C) = \frac{|\{(v, u, t) \in \mathcal{E}_{[t_s, t_e]} | v \in C, u \notin C\}|}{\min\{|\{(v, u, t) \in \mathcal{E}_{[t_s, t_e]} | v \in C\}|, |\{(v, u, t) \in \mathcal{E}_{[t_s, t_e]} | v \notin C\}|\}} \quad (19)$$

TD quantifies the internal cohesion of C by computing its average density. Higher TD values indicate stronger internal cohesion. TC measures the separation between C and the remainder of the graph. Lower TC values reflect better community isolation. Thus, an optimal community **maximizes TD** while **minimizing TC**.

Settings. We set the HM-Index partition coefficient $p = 4$ and leaf threshold $x = 40$ to balance construction efficiency and query speed. The learning rate is 0.001. AT-GNN samples 200 random vertices and 200 time intervals for training, while COCO runs for 150 epochs with early stopping. We apply 10 fine-tuning steps. During search, we extract a 5-hop subgraph per query vertex and set $\gamma = 1$ in Eq. 17. All baselines follow their published defaults. QTCS uses its standard settings, temporal k -core sets k to the query vertex core-ness, and label-dependent models (IACS [6], CS-TGN [11]) generate pseudo-labels via TransZero [35]. Each experiment averages results over 100 random single-vertex queries.

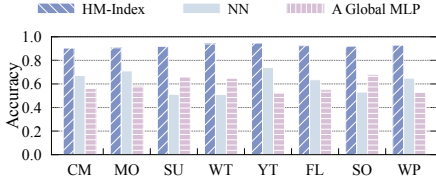
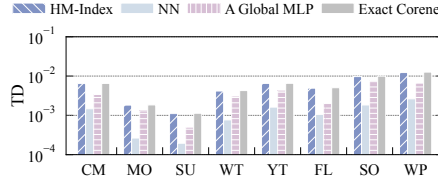
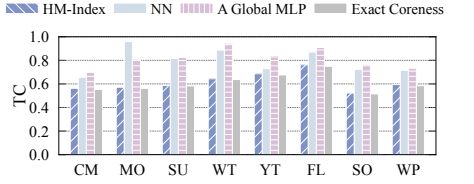
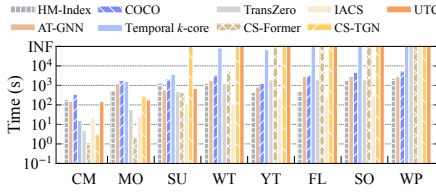
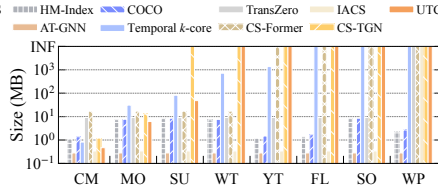
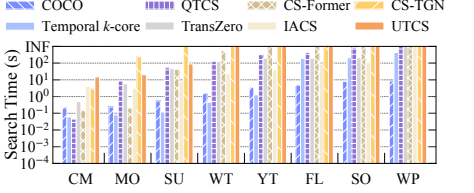
6.1 Effectiveness Evaluation

First, we evaluate the effectiveness of COCO, including both the overall framework and the HM-Index component.

Table 2: Community quality comparison for different methods

Datasets	CM		MO		SU		WT		YT		FL		SO		WP		Avg +/-	
Metrics	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC
COCO	0.0064	0.5625	0.0018	0.5715	0.0011	0.5872	0.0042	0.6441	0.0065	0.6862	0.0049	0.7624	0.0097	0.5225	0.0123	0.5947	-	-
Temporal k -core	0.0044	0.6771	0.0006	0.7292	0.0001	0.6762	0.0016	0.8304	0.0008	0.7443	0.0001	0.8263	0.0031	0.7231	0.0051	0.8308	-69.75%	+23.37%
QTCS	0.0048	0.6332	0.0011	0.6572	0.0006	0.6580	0.0022	0.7408	0.0028	0.8676	0.0033	0.8695	0.0045	0.7786	OOM	OOM	-43.22%	+20.59%
TransZero	0.0026	0.8433	0.0002	0.8999	9.32×10^{-5}	0.9293	0.0001	0.8838	0.0004	0.9650	0.0003	0.9865	7.02×10^{-5}	0.9780	OOM	OOM	-89.14%	+51.40%
CS-Former	0.0007	0.9037	0.0003	0.9490	8.05×10^{-6}	0.9660	0.0002	0.9388	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	-94.71%	+59.24%
IACS	0.0030	0.9500	0.0002	0.8340	0.0001	0.9895	0.0004	0.6850	1.98×10^{-7}	0.8785	8.04×10^{-8}	0.9872	1.03×10^{-6}	0.8942	OOM	OOM	-90.69%	+45.47%
CS-TGN	0.0024	0.9059	5.12×10^{-5}	0.7008	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	-78.64%	+41.84%
UTCS	0.0019	0.6876	0.0016	0.6678	0.0003	0.6481	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	-51.90%	+16.49%

* A community is considered to be of higher quality when it exhibits a **higher TD** and a **lower TC**.

**Figure 5: Coreness prediction accuracy****Figure 6: Effect of coreness prediction on TD****Figure 7: Effect of coreness prediction on TC****Figure 8: Training/Construction time****Figure 9: Model/Index Size****Figure 10: Search time**

Exp-1: Community Quality Comparison. We evaluate community quality for all methods using TD and TC metrics. Table 2 reports the results across all datasets. We can observe that COCO demonstrates consistent superiority, outperforming all competitors in both cohesion (TD) and separation (TC). For instance, compared with the non-learning method QTCS, COCO has +43.22% TD improvement and +20.59% TC improvement. compared with the learning method TransZero, COCO has +89.14% TD improvement, and +51.40% TC improvement. COCO’s dominance stems from its dual innovations: (1) HM-Index-derived coreness ensures cohesive community structures and (2) AT-GNN captures fine-grained evolutionary patterns. In contrast, methods such as temporal k -core and QTCS over-constrain communities via rigid k -core structures, excluding well-connected vertices. Additionally, learning-based approaches including TransZero, IACS, and CS-TGN fail to explicitly model structural cohesion, degrading performance.

Exp-2: Coreness Prediction Accuracy of HM-Index. To justify the hierarchical design of HM-Index, we compare it with two baselines: a global MLP trained on random intervals from the full graph lifespan, and a nearest-neighbor (NN) method that returns the exact coreness from the cached anchor interval with the largest temporal overlap (IoU) with the query. We measure accuracy by $1 - \text{MAPE}$, where $\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{c(v)_i - \hat{c}(v)_i}{c(v)_i} \right|$. As shown in Figure 5, HM-Index consistently achieves over 90% accuracy, while both baselines perform much worse. The global MLP lacks enough capacity to handle the topological changes caused by query intervals of different lengths. The NN method is also inaccurate because temporal k -core structures are highly sensitive to edge changes.

Even with large temporal overlap, missing a few critical edges can trigger cascading neighbor removals and break the core structure. HM-Index avoids these problems through its hierarchical design.

Exp-3: Effect of Coreness Accuracy on Community Quality. To evaluate how coreness prediction accuracy affects the final community quality, we compare the downstream TD and TC obtained using coreness values from HM-Index, NN, Global MLP, and exact coreness computation. As shown in Figures 6 and 7, NN and Global MLP lead to poor community quality across all datasets, yielding much lower TD and much higher TC. This is because their inaccurate coreness predictions substantially distort the AT-GNN feature space. In contrast, HM-Index produces results that are nearly identical to those obtained with exact coreness. These results suggest two conclusions. First, accurate coreness prediction is necessary for identifying cohesive communities. Second, the prediction accuracy of HM-Index is high enough to preserve strong downstream community quality, without incurring the heavy cost of online exact core decomposition.

6.2 Efficiency Evaluation

We evaluate the efficiency of COCO against other methods. In experiments, we mark the algorithm as INF if it is out of memory or its running time exceeds 10^5 seconds.

Exp-4: Offline Cost Comparison. We compare the offline cost of different methods, including training or index construction time and model or index size. The results are shown in Figures 8 and 9. **Training / Construction Time** (Figure 8). For temporal k -core, we report index construction time. For learning-based methods, including COCO, TransZero, CS-Former, IACS, and CS-TGN, we

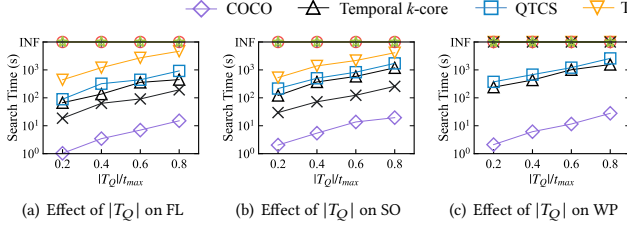
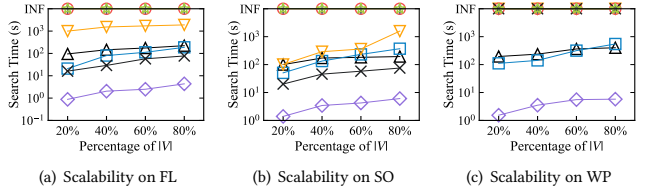
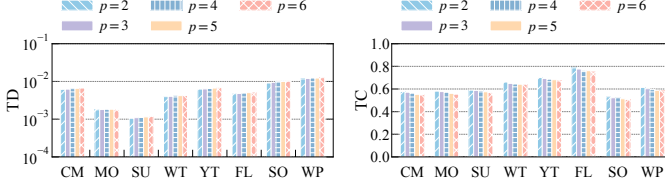
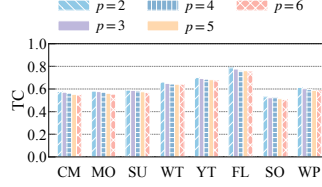
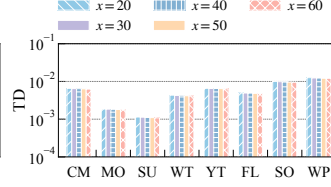
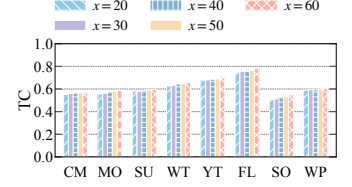
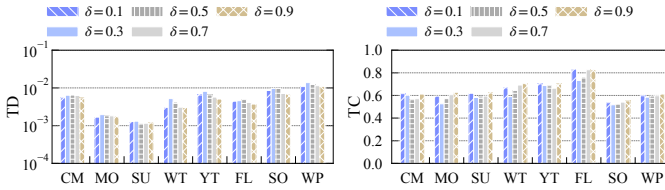
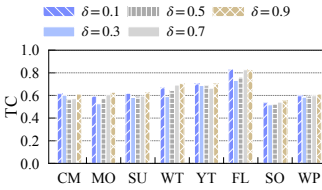
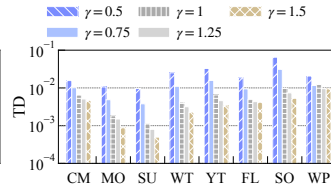
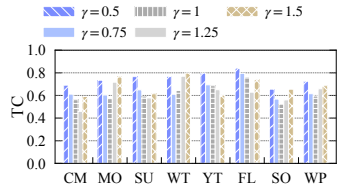
Figure 11: Effect of $|T_Q|$ 

Figure 12: Scalability

Figure 13: Effect of p on TDFigure 14: Effect of p on TCFigure 15: Effect of x on TDFigure 16: Effect of x on TCFigure 17: Effect of δ on TDFigure 18: Effect of δ on TCFigure 19: Effect of γ on TDFigure 20: Effect of γ on TC

report training time. COCO achieves training efficiency comparable to other learning-based methods, showing good scalability. In contrast, temporal k -core requires much longer construction time, especially on large datasets, because it must compute vertex coreness over all time intervals. **Model / Index Size** (Figure 9). COCO is also compact. Its AT-GNN is smaller than the models used by other learning-based methods. Although HM-Index adds storage for the tree structure and node-level MLPs, the overall memory cost remains low because these MLPs are lightweight. By contrast, temporal k -core uses much more memory, since it stores coreness values for all vertices over all time intervals, and its space cost grows quickly with graph size.

Exp-5: Online Search Time Comparison. COCO's search time includes the fine-tuning time. The results are shown in Figure 10. Compared with non-learning methods, COCO demonstrates substantially better online efficiency. Specifically, COCO is approximately two orders of magnitude faster than QTCS. Compared with temporal k -core, COCO is slower on small temporal graphs (i.e., CM, MO, SU, WT, and TY) but faster on large temporal graphs (i.e., FL, SO, and WP). The reason is that, on small temporal graphs, the temporal k -core index can be successfully constructed, allowing answers to be retrieved directly. However, on large temporal graphs, the temporal k -core index cannot be built due to its high construction and storage cost, and thus temporal k -core has to perform online decomposition, which significantly increases query time.

Exp-6: Effect of $|T_Q|$ on Search Time. We investigate the impact of the query time interval length, denoted as $|T_Q|$, on search efficiency by evaluating query intervals ranging from 20% to 80% of $t_{\max}$. Figure 11 presents the corresponding results on the FL and SO datasets. From these experiments, we obtain two key findings. First, all methods require longer search times as the query time interval increases, since broader temporal ranges involve processing a larger

number of vertices and edges. Second, COCO consistently outperforms the baseline methods by at least one order of magnitude, primarily due to its efficient fine-tuning mechanism.

Exp-7: Scalability. Figure 12 shows the scalability evaluation results of all methods on the FL and SO datasets. In this experiment, we randomly sampled vertices from the original graph to create graphs of varying sizes. As expected, the performance of all methods degrades as the graph size increases, since larger graphs usually contain larger communities. However, COCO maintains at least a $10\times$ speed advantage over all baselines. This demonstrates COCO's scalability for handling graphs of different scales.

6.3 Parameters Analysis

We analyze COCO's parameter sensitivity by varying several key parameters. For the HM-Index, we adjust two parameters: the partition coefficient p and the leaf node threshold x . For AT-GNN, we consider two parameters: δ , which adjusts the loss weights in Eq. (16), and γ , which controls the search process in Eq. (17).

Exp-8: Effect of Partition Coefficient p . The effect of the partition coefficient p in HM-Index is shown in Figure 13 and Figure 14. As p increases, community quality improves consistently on all datasets, with higher TD and lower TC. This is mainly because a larger p creates more branches in the hierarchical tree and produces finer time partitions. As a result, the regression model at each node can better capture temporal and structural patterns within each sub-interval, improving coreness prediction. Overall, HM-Index is robust to p , since the changes in community quality are limited. We set $p = 4$ by default because it balances prediction reliability and index complexity.

Exp-9: Effect of Leaf Node Threshold x . The effect of the leaf node threshold x on community quality is shown in Figure ?? . As

Table 3: Ablation study

Datasets	CM		MO		SU		WT		YT		FL		SO		WP		Avg +/-	
Metrics	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC
COCO (full)	0.0064	0.5625	0.0018	0.5715	0.0011	0.5872	0.0042	0.6441	0.0065	0.6862	0.0049	0.7624	0.0097	0.5225	0.0123	0.5947	-	-
w/o HM-Index (use single MLP)	0.0034	0.6982	0.0013	0.8010	0.0005	0.8240	0.0031	0.9407	0.0042	0.8292	0.0020	0.9095	0.0077	0.7652	0.0066	0.7325	-39.78%	+32.55%
w/o AT-GNN (use GAT)	0.0051	0.5935	0.0015	0.7685	0.0010	0.5953	0.0038	0.7887	0.0057	0.7803	0.0046	0.8742	0.0093	0.5777	0.0097	0.7629	-12.15%	+16.17%
w/o Adapter (use linear layer)	0.0057	0.6208	0.0016	0.6522	0.0011	0.6037	0.0038	0.6491	0.0065	0.6943	0.0048	0.8639	0.0084	0.5237	0.0110	0.6355	-7.52%	+6.21%
w/o DFT (use raw temporal feature)	0.0051	0.5935	0.0015	0.7685	0.0010	0.5953	0.0038	0.7887	0.0057	0.7803	0.0046	0.8742	0.0093	0.5777	0.0097	0.7629	-12.15%	+16.17%
w/o $\mathcal{L}_{Coh}$ (use $\mathcal{L}_{Temp}$ only)	0.0041	0.5856	0.0017	0.5959	0.0009	0.7158	0.0036	0.7176	0.0041	0.7662	0.0041	0.8366	0.0066	0.6392	0.0113	0.5952	-20.75%	+10.69%
w/o $\mathcal{L}_{Temp}$ (use $\mathcal{L}_{Coh}$ only)	0.0056	0.6655	0.0017	0.6958	0.0009	0.6470	0.0035	0.7731	0.0059	0.6938	0.0039	0.9249	0.0082	0.5892	0.0118	0.7635	-12.21%	+16.73%
w/o Coreness Feature (use Node2Vec)	0.0051	0.6101	0.0014	0.6565	0.0010	0.6401	0.0028	0.6955	0.0060	0.9055	0.0045	0.9482	0.0077	0.5794	0.0109	0.6431	-16.58%	+14.46%

shown in the figure, community quality slightly declines as x increases from 20 to 60 across all datasets. A smaller x yields finer temporal partitions in the HM-Index leaf nodes, allowing regression models to better capture short-term dynamics and structural variations in vertex coreness. In contrast, larger x leads to coarser intervals that may miss subtle temporal patterns, causing a minor accuracy drop. Nevertheless, community quality remains good. We choose $x = 40$ as the default to balance prediction performance and indexing efficiency.

Exp-10: Effect of δ . We evaluate the impact of δ , which balances cohesive loss and temporal link loss in Eq.(16). Figures 17 and 18 present the results. We observe an initial quality improvement (rising TD, falling TC) as δ increases to 0.5, followed by degradation at higher values. It is because excessive temporal loss weighting ($\delta > 0.5$) undermines structural cohesion. The results establish $\delta = 0.5$ as the optimal trade-off, simultaneously maximizing community cohesiveness and temporal consistency across most datasets.

Exp-11: Effect of γ . We study the effect of γ that controls the threshold decay rate in the search function (Eq.(17)) by varying values from 0.5 to 1.5. The results shown in Figures 19 and 20 reveal that increasing γ reduces TD by producing larger communities, while causes minor TC fluctuations due to community size’s limited impact on TC. Notably, COCO maintains robust performance even with the default $\gamma = 1$, demonstrating its stability across parameter choices.

6.4 Ablation Study

Exp-12: Ablation Study. We evaluate the main components of COCO, including the hierarchical model index, AT-GNN (with its adapter and frequency modeling), the loss functions, and the input features. The results are shown in Table 3. (1) Replacing the HM-Index with a single MLP causes a large performance drop, including an average 39.78% decrease in TD. This confirms that the hierarchical design is necessary for coreness prediction. (2) Replacing AT-GNN with GAT reduces quality by 12.15% in TD and 16.17% in TC, showing that AT-GNN better captures temporal community patterns. (3) To test the adaptive mechanism, we replace the whole adapter with a single linear layer, removing the query-context feature (s_Q) and the bottleneck structure. The decline in performance shows that simple dimension mapping cannot model query-specific graph dynamics well. (4) Using raw temporal features instead of DFT-based modeling gives results close to the GAT variant, which highlights the value of frequency-domain features for capturing interaction patterns. (5) Removing the cohesive loss $\mathcal{L}_{Coh}$ lowers TD by 20.75% on average. (6) Removing the temporal link loss $\mathcal{L}_{Temp}$ decreases TD by 12.21% and increases TC by 16.73%, indicating that

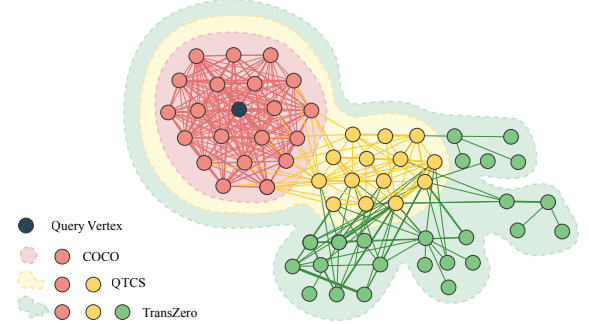


Figure 21: Case study on DBLP

both losses are important for structural and temporal consistency. (7) Replacing the structural coreness input feature with Node2Vec features also reduces community quality, suggesting that coreness is a better signal for finding cohesive subgraphs. Overall, every proposed component makes a clear contribution to COCO.

6.5 Case Study

Exp-13: Case Study. We conduct a case study on a temporal co-authorship network from DBLP [5], where vertices represent authors and timestamped edges denote publication collaborations. In the case study, we set the query time interval from 2020 to 2025. Figure 21 visualizes communities identified by COCO, QTCS, and TransZero, with edge thickness reflecting interaction frequency and the query author marked in black. COCO’s community, which consists of red vertices, demonstrates superior cohesion, exhibiting: (1) consistently high interaction frequencies among members, forming a tight collaboration network; (2) precise exclusion of weakly-connected authors (unlike QTCS’s yellow vertices with limited query author ties); and (3) avoidance of sparsely-interacting groups (unlike TransZero’s green vertices with few interactions). This real-world validation confirms COCO’s ability to identify authentic, tightly-knit academic communities.

7 Conclusion

This paper addresses the problem of time-interval-based community search. We propose COCO - a novel cohesiveness-aware learning framework that synergistically combines graph neural networks with structural constraints. COCO’s three-phase architecture (pre-training, fine-tuning, and searching) innovatively integrates two key components: (1) the HM-Index for hierarchical coreness prediction, and (2) AT-GNN for adaptive temporal pattern learning. COCO achieves both high-quality community finding and computational efficiency, as demonstrated through extensive experiments on real-world datasets.

References

- [1] Anonymous Author(s). 2026. COCO: A Cohesiveness-aware Learning Framework for Community Search over Temporal Graphs (Technical Report). <https://anonymous.4open.science/r/COCO-F183/COCO-SIGIR26-Technical%20Report.pdf>
- [2] Ali Behrouz, Farnoosh Hashemi, and Laks VS Lakshmanan. 2022. FirmTruss community search in multilayer networks. *Proceedings of the VLDB Endowment* 16, 3 (2022), 505–518.
- [3] Xueqin Chang, Qing Liu, Yunjun Gao, Baihua Zheng, Yi Cai, and Qing Li. 2025. The most influenced community search on social networks. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 2601–2614.
- [4] Lingyang Chu, Yanyan Zhang, Yu Yang, Lanjun Wang, and Jian Pei. 2019. Online density bursting subgraph detection from temporal graphs. *Proceedings of the VLDB Endowment* 12, 13 (2019), 2353–2365.
- [5] DBLP. 2024. <https://dblp.uni-trier.de/xml/>
- [6] Shuheng Fang, Kangfei Zhao, Yu Rong, Zhixun Li, and Jeffrey Xu Yu. 2024. Inductive attributed community search: to learn communities across graphs. *Proceedings of the VLDB Endowment* 17, 10 (2024), 2576–2589.
- [7] Yixiang Fang, Xin Huang, Lu Qin, Ying Zhang, Wenjie Zhang, Reynold Cheng, and Xuemin Lin. 2020. A survey of community search over big graphs. *The VLDB Journal* 29 (2020), 353–392.
- [8] Yixiang Fang, Zhongran Wang, Reynold Cheng, Hongzhi Wang, and Jiafeng Hu. 2018. Effective and efficient community search over large directed graphs. *IEEE Transactions on Knowledge and Data Engineering* 31, 11 (2018), 2093–2107.
- [9] Edoardo Galimberti, Alain Barrat, Francesco Bonchi, Ciro Cattuto, and Francesco Gullo. 2018. Mining (maximal) span-cores from temporal networks. In *Proceedings of the 27th ACM international Conference on Information and Knowledge Management*. 107–116.
- [10] Edoardo Galimberti, Martino Ciaperoni, Alain Barrat, Francesco Bonchi, Ciro Cattuto, and Francesco Gullo. 2020. Span-core decomposition for temporal networks: Algorithms and applications. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 15, 1 (2020), 1–44.
- [11] Farnoosh Hashemi, Ali Behrouz, and Milad Rezaei Hajidehi. 2023. Cs-tgn: Community search via temporal graph neural networks. In *Companion Proceedings of the ACM Web Conference 2023*. 1196–1203.
- [12] Xin Huang, Laks VS Lakshmanan, and Jianliang Xu. 2019. *Community search over big graphs*. Morgan & Claypool Publishers.
- [13] Yuli Jiang, Yu Rong, Hong Cheng, Xin Huang, Kangfei Zhao, and Junzhou Huang. 2022. Query driven-graph neural networks for community search: From non-attributed, attributed, to interactive attributed. *Proceedings of the VLDB Endowment* 15, 6 (2022), 1243–1255.
- [14] Zongli Jiang, Yirui Tan, Guoxin Chen, Fangda Guo, Jinli Zhang, and Xiaolu Bai. 2024. GNN-Based Persistent K-core Community Search in Temporal Graphs. In *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 569–578.
- [15] Wissam Khaoiuid, Marina Barsky, Venkatesh Srinivasan, and Alex Thomo. 2015. K-core decomposition of large networks on a single PC. *Proceedings of the VLDB Endowment* 9, 1 (2015), 13–23.
- [16] Jon M Kleinberg. 2000. Navigation in a small world. *Nature* 406, 6798 (2000), 845–845.
- [17] Ling Li, Siqiang Luo, Yuhai Zhao, Caihua Shan, Zhengkui Wang, and Lu Qin. 2023. COCLEP: Contrastive learning-based semi-supervised community search. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 2483–2495.
- [18] Ling Li, Yuhai Zhao, Yuan Li, Fazal Wahab, and Zhengkui Wang. 2022. The most active community search in large temporal graphs. *Knowledge-Based Systems* 250 (2022), 109101.
- [19] Rong-Hua Li, Jiao Su, Lu Qin, Jeffrey Xu Yu, and Qiangqiang Dai. 2018. Persistent community search in temporal networks. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 797–808.
- [20] Yuan Li, Jinsheng Liu, Huiqun Zhao, Jing Sun, Yuhai Zhao, and Guoren Wang. 2021. Efficient continual cohesive subgraph search in large temporal graphs. *World Wide Web* 24 (2021), 1483–1509.
- [21] Longlong Lin, Pingpeng Yuan, Rong-Hua Li, Chunxue Zhu, Hongchao Qin, Hai Jin, and Tao Jia. 2024. QTCs: Efficient Query-Centered Temporal Community Search. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1187–1199.
- [22] Qing Liu, Minjun Zhao, Xin Huang, Jianliang Xu, and Yunjun Gao. 2020. Truss-based community search over large directed graphs. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2183–2197.
- [23] Qing Liu, Yifan Zhu, Minjun Zhao, Xin Huang, Jianliang Xu, and Yunjun Gao. 2020. VAC: vertex-centric attributed community search. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 937–948.
- [24] Chengyang Luo, Qing Liu, Yunjun Gao, and Jianliang Xu. 2025. Synergetic Community Search over Large Multilayer Graphs. *Proceedings of the VLDB Endowment* 18, 5 (2025), 1412–1424.
- [25] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [26] Fragkiskos D Malliaros, Christos Giatsidis, Apostolos N Papadopoulos, and Michalis Vazirgiannis. 2020. The core decomposition of networks: Theory, algorithms and applications. *The VLDB Journal* 29, 1 (2020), 61–92.
- [27] Hongchao Qin, Rong-Hua Li, Guoren Wang, Lu Qin, Yurong Cheng, and Ye Yuan. 2019. Mining periodic cliques in temporal networks. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 1130–1141.
- [28] Hongchao Qin, Rong-Hua Li, Ye Yuan, Guoren Wang, Lu Qin, and Zhiwei Zhang. 2022. Mining bursting core in large temporal graphs. *Proceedings of the VLDB Endowment* (2022).
- [29] Hongchao Qin, Rong-Hua Li, Ye Yuan, Guoren Wang, Weihua Yang, and Lu Qin. 2020. Periodic communities mining in temporal networks: Concepts and algorithms. *IEEE Transactions on Knowledge and Data Engineering* 34, 8 (2020), 3927–3945.
- [30] Florian Schroff, Dmitry Kalenichenko, and James Philbin. 2015. Facenet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 815–823.
- [31] Arlei Silva, Ambuj Singh, and Ananthram Swami. 2018. Spectral algorithms for temporal graph cuts. In *Proceedings of the 2018 World Wide Web Conference*. 519–528.
- [32] Qing Sima, Jianke Yu, Xiaoyang Wang, Wenjie Zhang, Ying Zhang, and Xuemin Lin. 2025. Deep Overlapping Community Search via Subspace Embedding. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–26.
- [33] Longxu Sun, Xin Huang, Zheng Wu, and Jianliang Xu. 2024. Efficient Cross-layer Community Search in Large Multilayer Graphs. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2959–2971.
- [34] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- [35] Jianwei Wang, Kai Wang, Xuemin Lin, Wenjie Zhang, and Ying Zhang. 2024. Efficient unsupervised community search with pre-trained graph transformer. *Proceedings of the VLDB Endowment* 17, 9 (2024), 2227–2240.
- [36] Jianwei Wang, Kai Wang, Xuemin Lin, Wenjie Zhang, and Ying Zhang. 2024. Neural attributed community search at billion scale. *Proceedings of the ACM on Management of Data* 1, 4 (2024), 1–25.
- [37] Kai Wang, Wenjie Zhang, Xuemin Lin, Ying Zhang, Lu Qin, and Yuting Zhang. 2021. Efficient and effective community search on large-scale bipartite graphs. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. 85–96.
- [38] Kai Wang, Wenjie Zhang, Ying Zhang, Lu Qin, and Yuting Zhang. 2021. Discovering significant communities on bipartite graphs: An index-based approach. *IEEE Transactions on Knowledge and Data Engineering* 35, 3 (2021), 2471–2485.
- [39] Yuxiang Wang, Xiaoxuan Gou, Xiaoliang Xu, Yuxia Geng, Xiangyu Ke, Tianxing Wu, Zhiyuan Yu, Runhui Chen, and Xiangying Wu. 2024. Scalable community search over large-scale graphs based on graph transformer. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1680–1690.
- [40] Yue Wang, Xun Jian, Zhenhua Yang, and Jia Li. 2017. Query optimal k-plex based community in graphs. *Data Science and Engineering* 2 (2017), 257–273.
- [41] Yuxiang Wang, Shuzhan Ye, Xiaoliang Xu, Yuxia Geng, Zhenghe Zhao, Xiangyu Ke, and Tianxing Wu. 2024. Scalable community search with accuracy guarantee on attributed graphs. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2737–2750.
- [42] Junyong Yang, Ming Zhong, Yuanyuan Zhu, Tiejun Qian, Mengchi Liu, and Jeffrey Xu Yu. 2023. Scalable time-range k-core query on temporal graphs (full version). *arXiv preprint arXiv:2301.03770* (2023).
- [43] Junyong Yang, Ming Zhong, Yuanyuan Zhu, Tiejun Qian, Mengchi Liu, and Jeffrey Xu Yu. 2024. Evolution Forest Index: Towards Optimal Temporal k-Core Component Search via Time-Topology Isomorphic Computation. *Proceedings of the VLDB Endowment* 17, 11 (2024), 2840–2853.
- [44] Michael Yu, Dong Wen, Lu Qin, Ying Zhang, Wenjie Zhang, and Xuemin Lin. 2021. On querying historical k-cores. *Proceedings of the VLDB Endowment* (2021).
- [45] Yue Zhang, Yankai Chen, Yingli Zhou, Yucan Guo, Xiaolin Han, and Chenhao Ma. 2025. UTCS: Effective Unsupervised Temporal Community Search with Pre-training of Temporal Dynamics and Subgraph Knowledge. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2647–2651.
- [46] Yifei Zhang, Longlong Lin, Pingpeng Yuan, and Hai Jin. 2022. Significant engagement community search on temporal networks. In *International Conference on Database Systems for Advanced Applications*. Springer, 250–258.
- [47] Ming Zhong, Junyong Yang, Yuanyuan Zhu, Tiejun Qian, Mengchi Liu, and Jeffrey Xu Yu. 2024. A Unified and Scalable Algorithm Framework of User-Defined Temporal (k, X) -Core Query. *IEEE Transactions on Knowledge and Data Engineering* (2024).

A Proof of Lemma 4.2

PROOF OF LEMMA 4.2. Let $\hat{a}_k = \frac{1}{\sqrt{T}} \sum_{t=0}^{T-1} a_t e^{-2\pi i k t / T}$ denote the k -th coefficient of the unitary DFT of $\mathbf{a}$, and similarly for $\hat{b}_k$. Since the DFT is linear, $\hat{a}_k - \hat{b}_k$ is the k -th DFT coefficient of the difference sequence $\mathbf{a} - \mathbf{b}$.

By the reverse triangle inequality for complex modulus, we have $|\hat{a}_k| - |\hat{b}_k| \leq |\hat{a}_k - \hat{b}_k|$ for each frequency index k . Squaring and summing over the first d components yields $\|\phi_d(\mathbf{a}) - \phi_d(\mathbf{b})\|_2^2 = \sum_{k=0}^{d-1} (|\hat{a}_k| - |\hat{b}_k|)^2 \leq \sum_{k=0}^{d-1} |\hat{a}_k - \hat{b}_k|^2$.

Since $d \leq T$, the partial sum is bounded by the full sum: $\sum_{k=0}^{d-1} |\hat{a}_k - \hat{b}_k|^2 \leq \sum_{k=0}^{T-1} |\hat{a}_k - \hat{b}_k|^2$. By Parseval's identity for the unitary DFT, this equals $\|\mathbf{a} - \mathbf{b}\|_2^2 = \sum_{t=0}^{T-1} (a_t - b_t)^2$. Because $\mathbf{a}, \mathbf{b} \in \{0, 1\}^T$, each term $(a_t - b_t)^2 \in \{0, 1\}$, and exactly m terms are nonzero by assumption. Therefore $\|\mathbf{a} - \mathbf{b}\|_2^2 = m$.

Combining the above, we obtain $\|\phi_d(\mathbf{a}) - \phi_d(\mathbf{b})\|_2^2 \leq m$, which gives $\|\phi_d(\mathbf{a}) - \phi_d(\mathbf{b})\|_2 \leq \sqrt{m}$. $\square$

B Proof of Lemma 4.3

PROOF. Let the adapter formulation for a given query context be denoted as $\text{ADP}_Q(\mathbf{h}) = \sigma(\mathbf{W}_{\text{up}} \Phi_Q(\mathbf{W}_{\text{down}_1} \mathbf{h}, \mathbf{W}_{\text{down}_2} \mathbf{s}_Q))$, where Φ_Q abstracts the feature fusion module (Eq. 9) and σ is the ReLU activation function. Applying the multivariate chain rule, the Jacobian matrix of ADP_Q with respect to the input $\mathbf{h}$ is $\frac{\partial \text{ADP}_Q}{\partial \mathbf{h}} = \mathbf{D}_\sigma \mathbf{W}_{\text{up}} \mathbf{J}_\Phi \mathbf{W}_{\text{down}_1}$. Here, $\mathbf{D}_\sigma$ is a diagonal matrix containing the derivatives of the activation function, and $\mathbf{J}_\Phi$ is the Jacobian of the fusion module with respect to its first argument. According to the properties of matrix multiplication, the rank of a product cannot exceed the rank of any individual factor. Since the projection matrices are bounded by their dimensions, we have $\text{rank}(\mathbf{W}_{\text{down}_1}) \leq \dim_{\text{ADP}}$ and $\text{rank}(\mathbf{W}_{\text{up}}) \leq \dim_{\text{head}}$. Consequently, $\text{rank}(\frac{\partial \text{ADP}_Q}{\partial \mathbf{h}}) \leq \min\{\dim_{\text{ADP}}, \dim_{\text{head}}\} = \xi$, which proves the first part of the lemma.

To prove the second part, we construct a parameterization that exactly yields $\Delta_Q(\mathbf{h}) = \mathbf{A}_Q \mathbf{h} + \mathbf{b}_Q$. Assume the target affine transformation has a weight matrix $\mathbf{A}_Q$ with $\text{rank}(\mathbf{A}_Q) \leq \xi$. By definition, there exists a low-rank factorization $\mathbf{A}_Q = \mathbf{U}\mathbf{V}^T$, where $\mathbf{V}^T \in \mathbb{R}^{\xi \times \dim_{\text{in}}}$ and $\mathbf{U} \in \mathbb{R}^{\dim_{\text{out}} \times \xi}$. We can directly assign the adapter's projection weights such that $\mathbf{W}_{\text{down}_1} = \mathbf{V}^T$ and $\mathbf{W}_{\text{up}} = \mathbf{U}$, padding the remaining entries with zeros if ξ is strictly less than the bottleneck dimensions $\dim_{\text{ADP}}$ or $\dim_{\text{head}}$.

Finally, the trainable parameters within the attention-based fusion module Φ_Q can be configured to act as an identity mapping for the projected vertex feature $\mathbf{W}_{\text{down}_1} \mathbf{h}$, while mapping the conditioned query context $\mathbf{W}_{\text{down}_2} \mathbf{s}_Q$ to an additive offset vector $\tilde{\mathbf{b}}_Q$, such that $\mathbf{U}\tilde{\mathbf{b}}_Q = \mathbf{b}_Q$. Assuming the activation function σ operates in its linear regime for the target domain (e.g., non-negative orthant for ReLU), the adapter's computation simplifies to $\mathbf{W}_{\text{up}}(\mathbf{W}_{\text{down}_1} \mathbf{h} + \tilde{\mathbf{b}}_Q) = \mathbf{U}(\mathbf{V}^T \mathbf{h} + \tilde{\mathbf{b}}_Q)$. Expanding this expression perfectly recovers $\mathbf{A}_Q \mathbf{h} + \mathbf{b}_Q$, thus completing the proof. $\square$

C More Experimental Results

Exp-14: Effect of $|V_Q|$. The effect of the number of query vertices on community quality is shown in Figures 22 and 23. As $|V_Q|$ increases, we observe a slight downward trend in TD and a gradual

upward trend in TC across all datasets. Larger query sets typically require more intermediate vertices to maintain structural connectivity and cohesiveness, expanding community size. This slightly reduces internal interaction density and weakens external separation. Despite this, COCO maintains stable performance across query scales, demonstrating its robustness in multi-vertex community search.

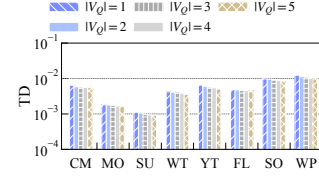


Figure 22: Effect of $|V_Q|$ on TD

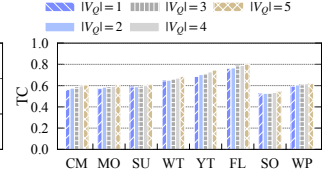


Figure 23: Effect of $|V_Q|$ on TC

Exp-16: Effect of # of Fine-tuning Steps. We explore the influence of the fine-tuning step number on performance of COCO. Specifically, we test 5, 10, 15, 20, and 25 steps. Figures 24 and 25 show the results. We have the following observations: the performance of COCO improves monotonically with additional steps; the most significant quality gains occur between 10-15 steps; and marginal returns diminish beyond this threshold. This suggests that while COCO benefits from sufficient adaptation (≥ 15 steps), excessive fine-tuning yields limited additional improvements, establishing an effective efficiency-quality tradeoff.

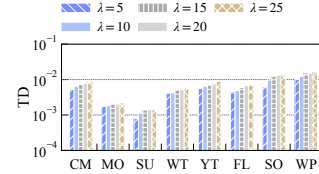


Figure 24: Effect of # of fine-tuning steps on TD

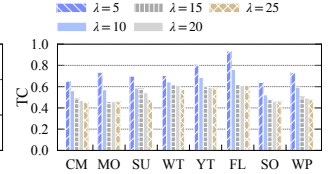


Figure 25: Effect of # of fine-tuning steps on TC

Exp-15: Effect of $|T_Q|$ on Prediction Accuracy of HM-Index. In this experiment, we evaluate the prediction accuracy of HM-Index by predicting vertex coreness across progressive time intervals (20%, 40%, 60%, and 80% of t_{max}). Results shown in Figure 26 demonstrate consistent performance above 90% accuracy across all datasets and interval lengths. This robustness stems from HM-Index's hierarchical architecture. COCO initiates its search from the root node, traverses through the HM-Index to identify nodes encompassing the query time interval. The models along the search path collectively refine coreness predictions.

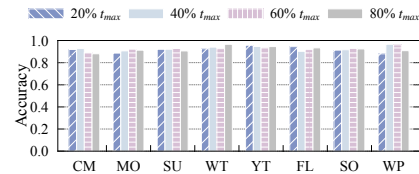


Figure 26: Effect of $|T_Q|$ on prediction accuracy